



Teoria e Aplicações de Inteligência Computacional I

Aula 5 – Introdução aos Fundamentos de Inteligência Computacional

Sistemas ANFIS (*Adaptative Network-based Fuzzy Inference Systems*)

Agenda da Aula



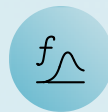
Cap. 1 — Do Neurônio Biológico ao ANFIS

Analogia biológica · Neurônio artificial · Forward Pass · Backward Pass



Cap. 2 — Arquitetura ANFIS

Regras Fuzzy Sugeno · Estrutura em 5 Camadas · Parâmetros de Premissa e Consequente



Cap. 3 — Mínimos Quadrados

Motivação · Função de Custo $J(a,b)$ · Equações Normais · Solução Analítica



Cap. 4 — Algoritmo de Aprendizado Híbrido

Por que não usar só Backpropagation? · Conexão LSE ↔ ANFIS · Passo Forward (LSE) · Passo Backward (Gradiente)



Cap. 5 — Backpropagation no ANFIS

Regra da Cadeia · $\partial E / \partial \hat{z}$ · $\partial \hat{z} / \partial w$ · $\partial w / \partial \mu_A$ · $\partial \mu_A / \partial c$ · Função Sino Generalizada



Cap. 6 — Épocas e Variantes

Nova Época · Ciclo Forward → Backward · ANFIS Clássico (Batch) · ANFIS Online / Estocástico

Do Neurônio Biológico ao Artificial

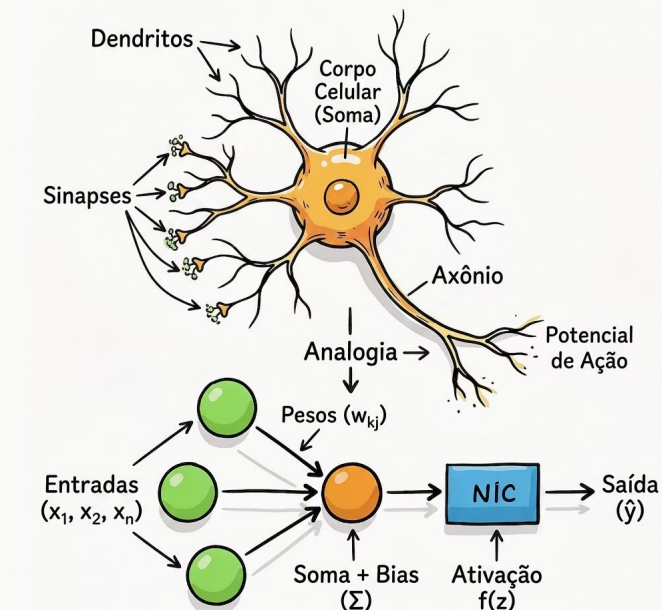


Figura 1. Analogia entre o neurônio biológico e o neurônio artificial (perceptron).

Dendritos →
Entradas ($x_1 \dots x_n$)

Sinais recebidos do
ambiente.

Sinapses → Pesos
(w_{kj})

Força de cada
conexão.

Corpo Celular →
Soma + Bias (z)

$$v_k = \sum w_{kj} x_j + b_k$$

Potencial de Ação
→ Ativação $f(z)$

$$y_k = \varphi(v_k)$$

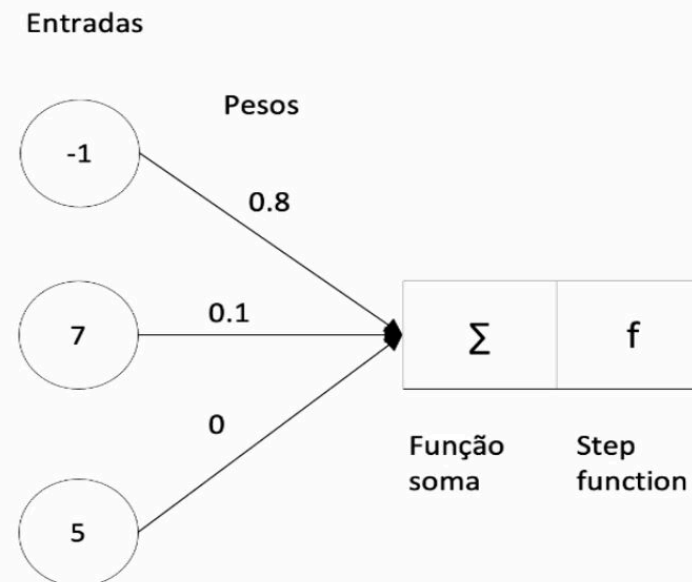


Figura 2. Modelo matemático do neurônio artificial.

Axônio → Saída (y)

Propaga o resultado para a próxima camada.

Forward Pass — Propagação Direta

No forward pass, os sinais são propagados da camada de entrada para a saída. Cada neurônio calcula uma soma ponderada das entradas, adiciona um bias e aplica uma função de ativação $\sigma(z)$.

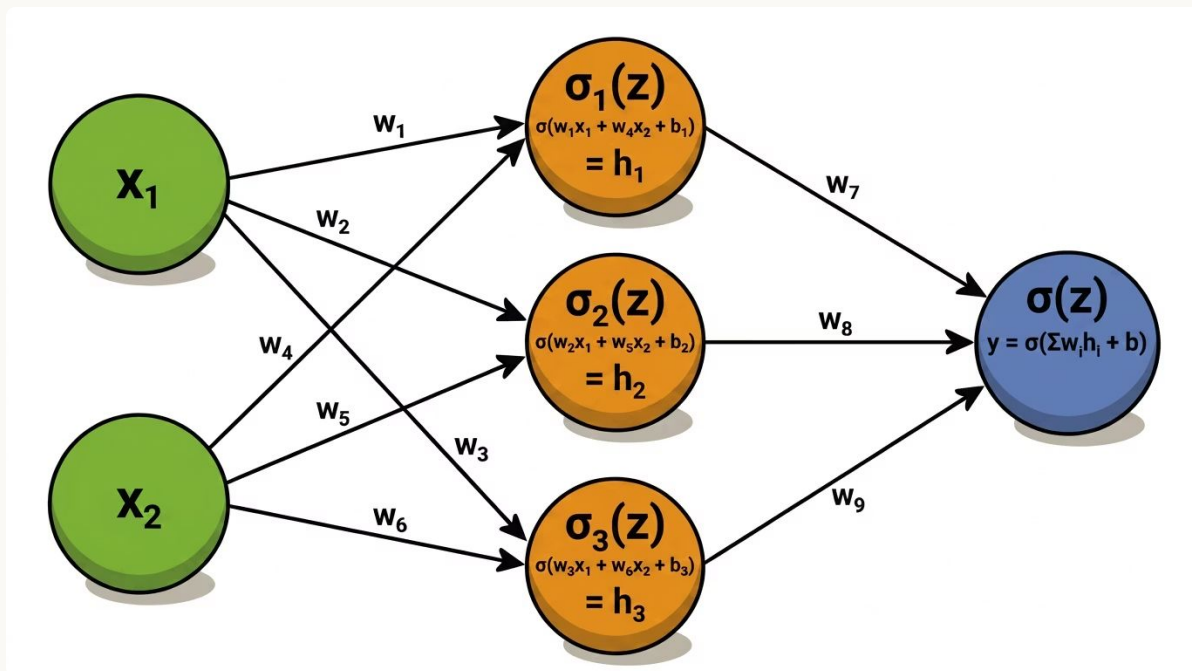


Figura 3. Rede neural com forward pass — propagação direta dos sinais.

Camada de Entrada (x_1, x_2)

Sinais crísp do ambiente. Cada entrada é conectada a todos os neurônios da camada oculta com pesos $w_1 \dots w_6$.

Camada Oculta (h_1, h_2, h_3)

Cada neurônio calcula:

$$h_i = \sigma(z_i) = \sigma(w \cdot x_1 + w \cdot x_2 + b_i)$$

Pesos ($w_1 \dots w_9$)

Parâmetros ajustáveis que determinam a força de cada conexão entre neurônios.

Camada de Saída $\sigma(z)$

Agrega as saídas da camada oculta:

$$y = \sigma \left(\sum_i w_i h_i + b \right)$$

Função de Ativação $\sigma(z)$

Introduz não-linearidade. Exemplos: Sigmoid, ReLU, Tanh. Permite à rede aprender padrões complexos.

Backward Pass — Retropropagação do Erro

No backward pass, o erro é propagado da saída para as camadas anteriores. Os pesos são atualizados proporcionalmente ao gradiente do erro, usando a regra delta e a taxa de aprendizado η .

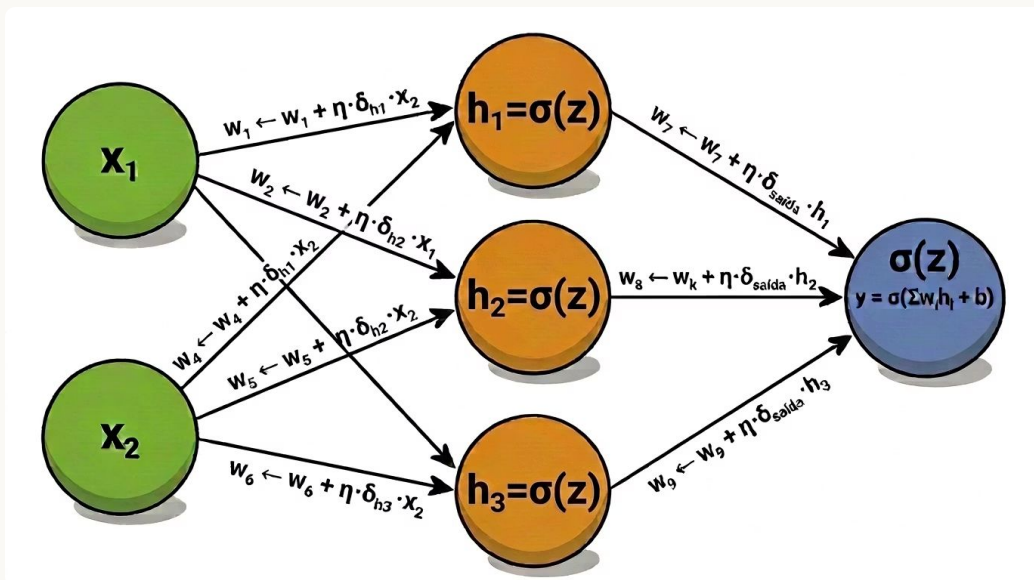


Figura 4. Rede neural com backward pass — retropropagação do erro e atualização dos pesos.

Erro na Saída ($\delta_{saída}$)

Calculado como a diferença entre o valor desejado e o obtido: $\delta_{saída} = (d - y) \cdot \sigma'(z)$

Erro na Camada Oculta (δ_{h_i})

Propagado de volta: $\delta_{h_i} = \sigma'(z) \cdot \sum (w_i \cdot \delta_{saída})$

Atualização dos Pesos (Camada Oculta → Saída)

$$w_k \leftarrow w_k + \eta \cdot \delta_{saída} \cdot h_i$$

Atualização dos Pesos (Entrada → Oculta)

$$w_j \leftarrow w_j + \eta \cdot \delta_{h_i} \cdot x_j$$

Taxa de Aprendizado (η)

Controla o tamanho do passo na atualização. Valores típicos: 0.001 a 0.1.

O Pipeline do Sistema de Inferência Fuzzy (FIS) – Mandani

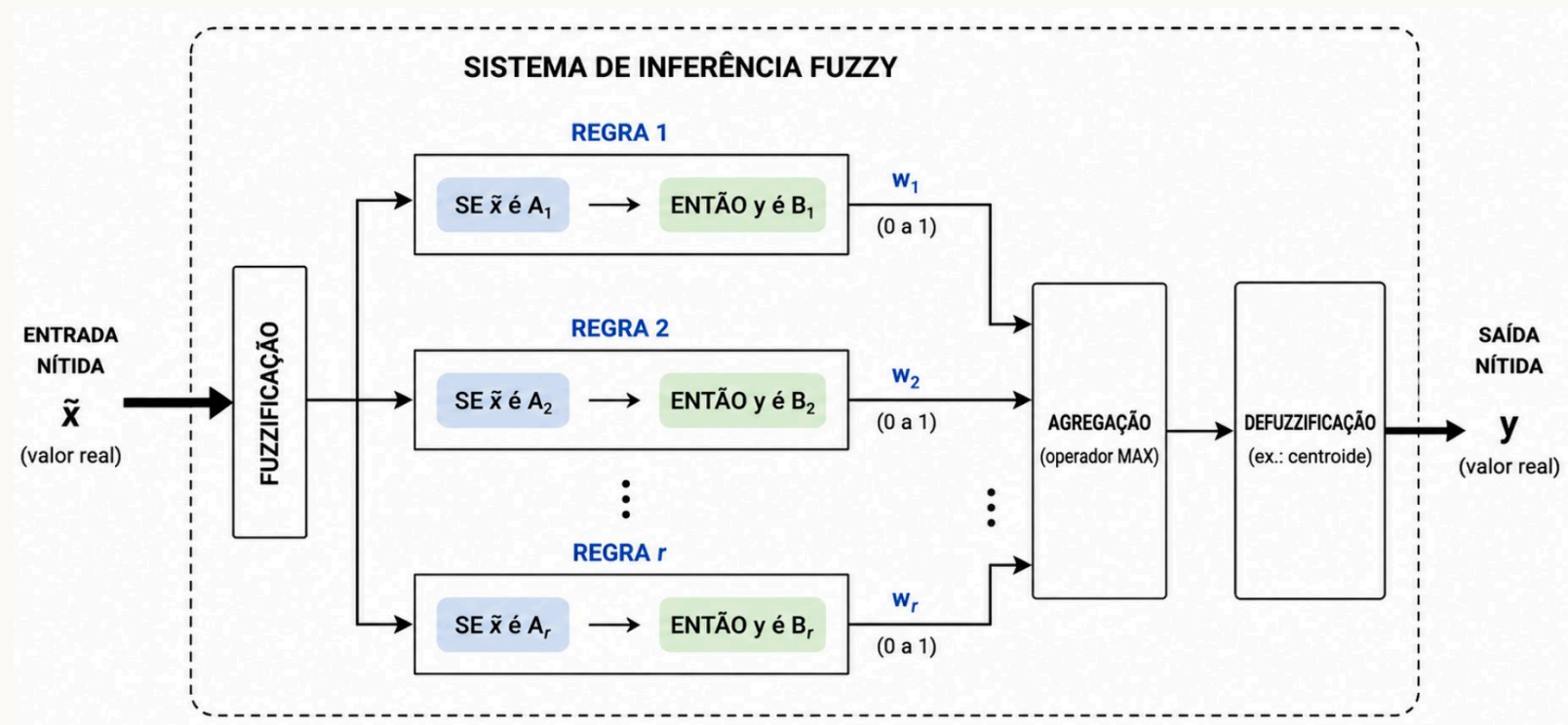


Figura 5. Pipeline do Sistema de Inferência Fuzzy Mamdani.

① Entradas

Sinais do mundo real (valores crisp) vindos de sensores.

② Fuzzificação

Converte valores crisp em graus de pertinência usando as MFs dos conjuntos fuzzy.

③ Base de Regras + Inferência

Regras "se-então" ativadas com pesos $w_1, w_2, \dots, w_r$ pelo mecanismo de raciocínio fuzzy.

④ Agregação

Combina as saídas de todas as regras em um único conjunto fuzzy agregado (operador MAX).

⑤ Defuzzificação

Transforma o conjunto fuzzy agregado em um valor crisp utilizável.

⑥ Saída

Valor final y (crisp) enviado ao sistema ou atuador.

Raciocínio Fuzzy Sugeno — Exemplo com 2 Regras

Para cada par de entradas (x, y) , as funções de pertinência A_1, A_2, B_1, B_2 determinam os graus de ativação w_1 e w_2 de cada regra. A saída final f é a média ponderada normalizada das saídas individuais.

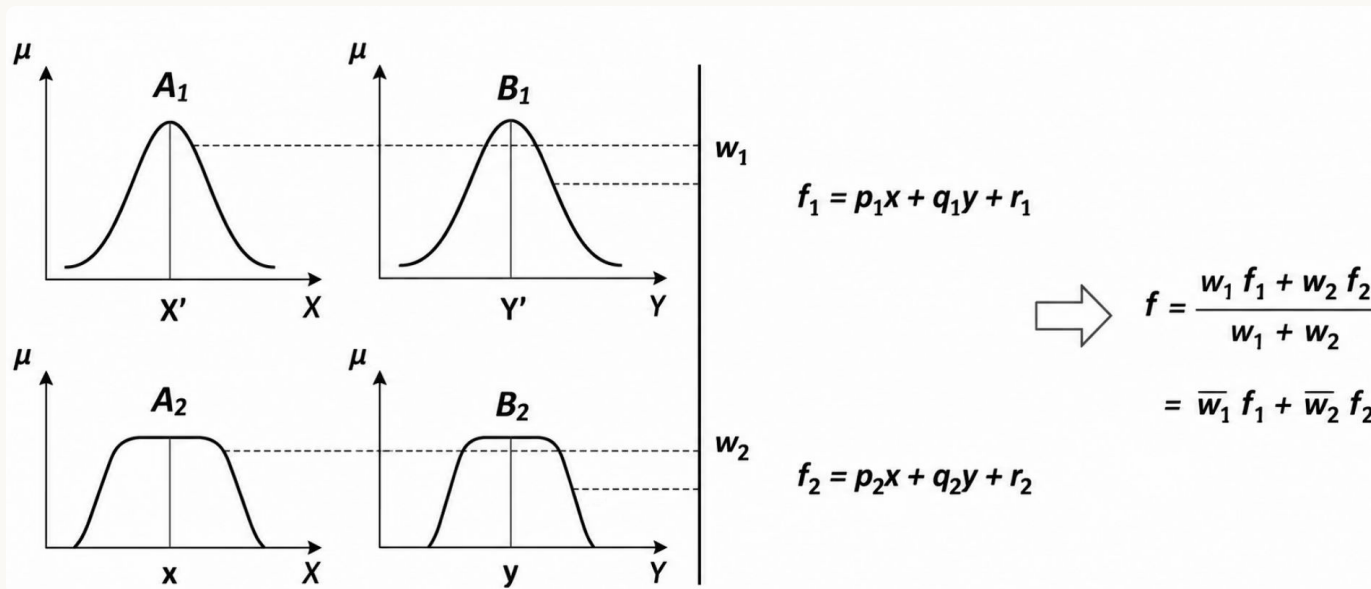


Figura 6. Raciocínio fuzzy Sugeno de primeira ordem com duas regras.

Regra 1 (w_1)

Se x é A_1 e y é B_1

$$f_1 = p_1x + q_1y + r_1$$

$$w_1 = \mu_{A_1}(x) \cdot \mu_{B_1}(y)$$

Saída Final (f)

$$f = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2} = \bar{w}_1 f_1 + \bar{w}_2 f_2$$

Média ponderada normalizada

Regra 2 (w_2)

Se x é A_2 e y é B_2

$$f_2 = p_2x + q_2y + r_2$$

$$w_2 = \mu_{A_2}(x) \cdot \mu_{B_2}(y)$$

Sistemas Neuro-Fuzzy – Aprendizado Híbrido

Sistemas neuro-fuzzy são um tipo de sistema inteligente híbrido, combinando as melhores características das redes neurais e da lógica fuzzy para o design de soluções robustas.



Redes Neurais Artificiais

Reconhecem padrões e se adaptam a ambientes em mudança.



Sistemas de Inferência Fuzzy (FIS)

Incorporam conhecimento humano e realizam inferência e tomada de decisão.

Motivação: Aprender a Relação entre Variáveis

Como estimar a saída de um sistema a partir de dados experimentais? O ANFIS aprende essa relação automaticamente, ajustando suas regras fuzzy e parâmetros com base nos dados observados.

Exemplo: Controle de Velocidade de um Ventilador

Entradas

Temperatura (T) e Umidade (U) – medidas do ambiente.

Saída

Velocidade do ventilador (V) – valor a ser estimado pelo modelo.

Objetivos

- A partir de pares experimentais $(T, U) \rightarrow V$, o **ANFIS** aprende a função que mapeia entradas à saída desejada.
- O objetivo não é apenas definir regras fuzzy, mas permitir que o modelo aprenda e ajuste essas regras a partir dos dados.

Dados Experimentais

Temperatura	Umidade	Velocidade Referência	Velocidade Medida
20 °C	40 %	20 %	20%
25 °C	45 %	35 %	32%
30 °C	50 %	55 %	58%
35 °C	60 %	80 %	75%
40 °C	70 %	100 %	92%

Tabela 1. Dados experimentais de treinamento para o controle do ventilador.

Arquitetura ANFIS

Para simplificar, considera-se um sistema de inferência fuzzy com duas entradas x e y e uma saída z . Para um modelo fuzzy Sugeno de primeira ordem [Jang, 1993], um conjunto de regras típico com duas regras fuzzy if-then é:

Regra 1

Se x é A_1 e y é B_1 , então $f_1 = p_1x + q_1y + r_1$

Regra 2

Se x é A_2 e y é B_2 , então $f_2 = p_2x + q_2y + r_2$

Onde A_1, A_2, B_1, B_2 são **conjuntos fuzzy** (parâmetros de premissa) e $p_1, q_1, r_1, p_2, q_2, r_2$ são **parâmetros lineares da consequente** (parâmetros de consequente).

Diagrama da Arquitetura ANFIS

A rede ANFIS equivale funcionalmente a um sistema de inferência fuzzy Sugeno de primeira ordem. Os parâmetros são divididos em dois conjuntos: **parâmetros de premissa** (não-lineares, Camadas 1) e **parâmetros de consequente** (lineares, Camada 4), otimizados pelo algoritmo híbrido.

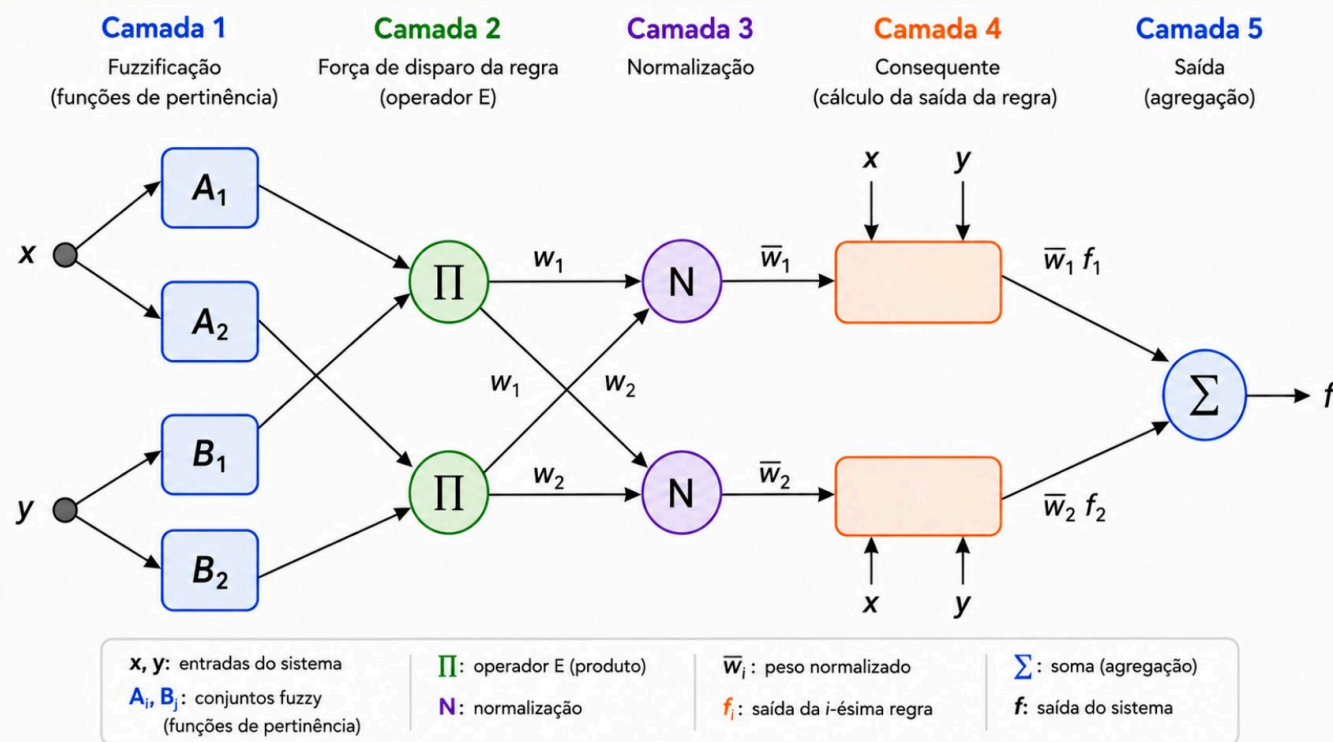


Figura 7. Diagrama da arquitetura ANFIS com 5 camadas.

Parâmetros de Premissa (não-lineares)

Camada 1: parâmetros das funções de pertinência $\{a_i, b_i, c_i\}$. Otimizados por gradiente descendente.

Parâmetros de Consequente (lineares)

Camada 4: coeficientes $\{p_i, q_i, r_i\}$. Otimizados por mínimos quadrados (LSE).

Algoritmo Híbrido

Passo forward: LSE para consequentes. Passo backward: gradiente para premissas.

Estrutura em 5 Camadas da Rede ANFIS

Camada 1 — Fuzzificação

Nós adaptativos. Cada nó computa o grau de pertinência das entradas:

$$O_i^1 = \mu_{A_i}(x) \text{ ou } \mu_{B_i}(y)$$

Camada 2 — Força de Disparo

Nós fixos (Π). Calcula o produto das pertinências:

$$w_1 = \mu_{A_1}(x) \cdot \mu_{B_1}(y), \quad w_2 = \mu_{A_2}(x) \cdot \mu_{B_2}(y)$$

Camada 3 — Normalização

Nós fixos (N). Normaliza as forças de disparo:

$$\bar{w}_i = \frac{w_i}{w_1 + w_2}$$

Camada 4 — Consequente

Nós adaptativos. Saída de cada nó:

$$\bar{w}_i f_i = \bar{w}_i (p_i x + q_i y + r_i)$$

Camada 5 — Saída

Nó fixo (Σ). Soma todas as saídas:

$$f = \sum_i \bar{w}_i f_i = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2}$$

Camada 1 — Fuzzificação (Nós Adaptativos)

Todo nó i nesta camada é um nó adaptativo com a seguinte função de nó:

$$O_{1,i} = \mu_{A_i}(x), \quad \text{para } i = 1, 2$$

$$O_{1,i} = \mu_{B_{i-2}}(y), \quad \text{para } i = 3, 4$$

onde x (ou y) é a entrada do nó i e A_i (ou B_{i-2}) é um rótulo linguístico (como "pequeno" ou "grande") associado a este nó. Em outras palavras, $O_{1,i}$ é o grau de pertinência de um conjunto fuzzy A ($= A_1, A_2, B_1$ ou B_2) e especifica o grau em que a entrada x (ou y) satisfaz o quantificador A .

Função de Pertinência — Sino Generalizado

$$\mu_A(x) = \frac{1}{1 + \left| \frac{x - c_i}{a_i} \right|^{2b_i}}$$

onde $\{a_i, b_i, c_i\}$ é o conjunto de parâmetros. À medida que os valores desses parâmetros mudam, a função sino varia de acordo, exibindo assim várias formas de funções de pertinência para o conjunto fuzzy A . Os parâmetros nesta camada são denominados **parâmetros de premissa**.

Saída $O_{1,i}$

Grau de pertinência μ da entrada x ou y nos conjuntos fuzzy A_1, A_2, B_1, B_2 .

Parâmetros de Premissa $\{a_i, b_i, c_i\}$

Controlam a forma da função de pertinência. São ajustados durante o treinamento pelo algoritmo híbrido.

Camadas 2 a 5 — Da Força de Disparo à Saída

Camada 2 — Força de Disparo (Nó Fixo Π)

Todo nó nesta camada é um nó fixo rotulado Π , cuja saída é o produto de todos os sinais de entrada:

$$O_{2,i} = w_i = \mu_{A_i}(x) \mu_{B_i}(y), \quad i = 1, 2$$

A saída de cada nó representa a força de disparo de uma regra. Em geral, qualquer operador T-norma que realize o AND fuzzy pode ser usado como função de nó nesta camada.

Camada 3 — Normalização (Nó Fixo N)

O i -ésimo nó calcula a razão entre a força de disparo da i -ésima regra e a soma das forças de disparo de todas as regras:

$$O_{3,i} = \bar{w}_i = \frac{w_i}{w_1 + w_2}, \quad i = 1, 2$$

Por conveniência, as saídas desta camada são chamadas de forças de disparo normalizadas.

Camada 4 — Consequente (Nó Adaptativo)

Todo nó i nesta camada é um nó adaptativo com a seguinte função de nó:

$$O_{4,i} = \bar{w}_i f_i = \bar{w}_i (p_i x + q_i y + r_i)$$

onde $\bar{w}_i$ é a força de disparo normalizada da Camada 3 e $\{p_i, q_i, r_i\}$ é o conjunto de parâmetros deste nó. Os parâmetros nesta camada são denominados parâmetros de consequente.

Camada 5 — Saída (Nó Fixo Σ)

O único nó nesta camada é um nó fixo rotulado Σ , que computa a saída geral como a soma de todos os sinais de entrada:

$$\text{saída geral} = O_{5,1} = \sum_i \bar{w}_i f_i = \frac{\sum_i w_i f_i}{\sum_i w_i}$$

Algoritmo de Aprendizado Híbrido

Da arquitetura ANFIS mostrada na Figura 7, observa-se que quando os valores dos parâmetros de premissa são fixos, a saída geral pode ser expressa como uma combinação linear dos parâmetros de consequente. Em símbolos, a saída f pode ser reescrita como:

$$f = \frac{w_1}{w_1 + w_2} f_1 + \frac{w_2}{w_1 + w_2} f_2 = \bar{w}_1(p_1x + q_1y + r_1) + \bar{w}_2(p_2x + q_2y + r_2) = (\bar{w}_1x)p_1 + (\bar{w}_1y)q_1 + \bar{w}_1r_1 + (\bar{w}_2x)p_2 + (\bar{w}_2y)q_2 + \bar{w}_2r_2$$

que é linear nos parâmetros de consequente p_1, q_1, r_1, p_2, q_2 e r_2 . Desta observação, temos:

S — Conjunto Total de Parâmetros

Conjunto completo de todos os parâmetros da rede ANFIS, composto pela união de S_1 e S_2 .

S_1 — Parâmetros de Premissa (não-lineares)

Parâmetros das funções de pertinência da Camada 1: $\{a_i, b_i, c_i\}$. Otimizados por gradiente descendente no passo *backward*.

S_2 — Parâmetros de Consequente (lineares)

Coefficientes da saída Sugeno da Camada 4: $\{p_i, q_i, r_i\}$. Otimizados por mínimos quadrados (LSE) no passo *forward*.

Ideia Central

No exemplo do ventilador, fixando os parâmetros de premissa, a saída V pode ser expressa como combinação linear dos parâmetros de consequente — o que torna o LSE aplicável diretamente. Essa é a chave do algoritmo híbrido.

Ideia Central

Dados de Treinamento $(x, y) \rightarrow f^*$

Cada par de entrada-saída observado impõe uma equação linear nos parâmetros de consequente $\{p_i, q_i, r_i\}$, quando os parâmetros de premissa estão fixos.

Separação de Papéis

Os parâmetros de premissa (não-lineares) definem a forma das funções de pertinência. Os parâmetros de consequente (lineares) definem a saída de cada regra. Essa separação permite usar métodos distintos e mais eficientes para cada grupo.

Convergência mais Rápida

O algoritmo híbrido converge significativamente mais rápido do que o gradiente descendente puro, pois o LSE resolve de forma ótima a parte linear do problema a cada época.

Fluxo do Algoritmo Híbrido

Passo Forward ($\rightarrow$)

Os parâmetros de premissa são fixos. As forças de disparo normalizadas $\bar{w}_i$ são calculadas. Os parâmetros de consequente $\{p_i, q_i, r_i\}$ são identificados **pelo método dos mínimos quadrados (LSE)**.

$$\min_{p,q,r} \sum_k \left(f_k^* - \hat{f}_k \right)^2$$

Passo Backward ($\leftarrow$)

Os parâmetros de consequente são fixos. O erro é retropropagado pela rede. Os parâmetros de premissa $\{a_i, b_i, c_i\}$ são atualizados por gradiente descendente.

$$a_i \leftarrow a_i - \eta \frac{\partial E}{\partial a_i}$$

Por que não usar apenas Backpropagation?

Embora possamos aplicar a retropropagação (backpropagation) ou o método do gradiente descendente para identificar os parâmetros em uma rede adaptativa, esse método de otimização simples geralmente leva muito tempo até convergir.

Problema do Gradiente Descendente Puro

Método simples, mas lento. Quando aplicado sozinho à identificação de todos os parâmetros do ANFIS, o tempo de convergência é elevado, tornando o treinamento ineficiente.

Observação Chave

A saída de uma rede adaptativa (assumindo que há apenas uma) é linear em alguns de seus parâmetros. Esses parâmetros lineares podem ser identificados de forma muito mais eficiente pelo método dos mínimos quadrados (LSE).

Regra de Aprendizado Híbrido [Jang, 1993]

Combina o gradiente descendente (SD) com o estimador de mínimos quadrados (LSE) para identificação rápida dos parâmetros. Referências: [1].

Resultado

O algoritmo híbrido converge muito mais rapidamente do que o gradiente descendente puro, aproveitando a estrutura linear dos parâmetros de consequente para aplicar o LSE de forma ótima a cada época de treinamento.

Mínimos Quadrados — Fundamento Matemático do ANFIS

Antes de aprofundar nas partes centrais dos sistemas neuro-fuzzy, introduzimos uma classe de métodos padrão de mínimos quadrados para modelos lineares em identificação de sistemas. Os métodos de mínimos quadrados são ferramentas matemáticas poderosas e bem desenvolvidas, propostas e utilizadas em diversas áreas há décadas, incluindo controle adaptativo, processamento de sinais e estatística.

Ferramentas Essenciais e Indispensáveis

Os métodos de mínimos quadrados continuam sendo ferramentas essenciais e indispensáveis para a construção de modelos matemáticos lineares, mesmo nos dias atuais.

Extensão para Modelos Não-Lineares

Os mesmos conceitos fundamentais podem ser estendidos para modelos não-lineares. Assim, basta saber que esses métodos lineares fornecem a base matemática mais importante para resolver problemas de modelagem neuro-fuzzy.

Aplicações

Controle adaptativo, processamento de sinais, estatística, identificação de sistemas e modelagem neuro-fuzzy (ANFIS).

Problema dos Mínimos Quadrados (LSE)

Dado um conjunto de pares (x_k, f_k^*) , encontrar os parâmetros θ que minimizam o erro quadrático total:

$$\min_{\theta} \sum_{k=1}^N \left(f_k^* - \hat{f}(x_k, \theta) \right)^2$$

Por que é central no ANFIS?

Quando os parâmetros de premissa estão fixos, a saída do ANFIS é linear nos parâmetros de consequente $\theta = \{p_i, q_i, r_i\}$. **O LSE resolve esse subproblema de forma exata e eficiente a cada passo forward do algoritmo híbrido.**

Método dos Mínimos Quadrados

O objetivo é encontrar a reta que melhor representa um conjunto de dados observados, minimizando a soma dos erros ao quadrado entre os valores reais e os valores preditos pelo modelo.

Formulação do Problema

O modelo linear é dado por:

$$\hat{y}_i = ax_i + b$$

Para cada ponto observado (x_i, y_i) , o erro residual é:

$$e_i = y_i - \hat{y}_i = y_i - (ax_i + b)$$

Função de Custo $J(a, b)$

$$J(a, b) = \sum_{i=1}^n [y_i - (ax_i + b)]^2$$

Busca-se encontrar os valores de a e b que minimizam $J(a, b)$.

Por que elevar ao quadrado?

Evita Cancelamento

Erros positivos e negativos não se anulam mutuamente, garantindo que todos os desvios contribuam para a medida total de erro.

Penaliza Erros Grandes

Ao elevar ao quadrado, erros maiores recebem peso proporcionalmente maior, tornando o modelo mais sensível a desvios significativos.

Solução Analítica Exata

A função quadrática é convexa e diferenciável, permitindo encontrar o mínimo global de forma analítica, igualando as derivadas parciais a zero:

$$\frac{\partial J}{\partial a} = 0, \quad \frac{\partial J}{\partial b} = 0$$

Derivada de J em relação a a

Para encontrar o valor de a que minimiza J(a, b), aplicamos a regra da cadeia à função de custo:

$$J(a, b) = \sum_{i=1}^n [y_i - (ax_i + b)]^2$$

Aplicando a Regra da Cadeia

Derivando J em relação a a:

$$\frac{\partial J}{\partial a} = \sum_{i=1}^n 2 [y_i - (ax_i + b)] \cdot \frac{\partial}{\partial a} [y_i - (ax_i + b)]$$

Como a derivada interna é:

$$\frac{\partial}{\partial a} [y_i - (ax_i + b)] = -x_i$$

Substituindo:

$$\frac{\partial J}{\partial a} = -2 \sum_{i=1}^n x_i [y_i - (ax_i + b)]$$

Condição de Mínimo

No ponto de mínimo, igualamos a derivada a zero:

$$-2 \sum_{i=1}^n x_i [y_i - (ax_i + b)] = 0$$

Simplificando (dividindo por -2):

$$\sum_{i=1}^n x_i [y_i - (ax_i + b)] = 0$$

Distribuindo a soma:

$$\sum_{i=1}^n x_i y_i - a \sum_{i=1}^n x_i^2 - b \sum_{i=1}^n x_i = 0$$

Reorganizando, obtemos a primeira equação normal:

$$a \sum x_i^2 + b \sum x_i = \sum x_i y_i$$

Derivada de J em relação a b

De forma análoga, derivamos J em relação a b para obter a segunda equação normal.

Aplicando a Regra da Cadeia

Derivando J em relação a b:

$$\frac{\partial J}{\partial b} = \sum_{i=1}^n 2[y_i - (ax_i + b)] \cdot \frac{\partial}{\partial b} [y_i - (ax_i + b)]$$

Como a derivada interna é:

$$\frac{\partial}{\partial b} [y_i - (ax_i + b)] = -1$$

Substituindo:

$$\frac{\partial J}{\partial b} = -2 \sum_{i=1}^n [y_i - (ax_i + b)]$$

Condição de Mínimo

No ponto de mínimo, igualamos a zero:

$$-2 \sum_{i=1}^n [y_i - (ax_i + b)] = 0$$

Simplificando (dividindo por -2):

$$\sum_{i=1}^n [y_i - (ax_i + b)] = 0$$

Distribuindo a soma:

$$\sum y_i - a \sum x_i - nb = 0$$

Reorganizando, obtemos a segunda equação normal:

$$a \sum x_i + nb = \sum y_i$$

Sistema de Equações Normais

Derivando J em relação a a e b e igualando a zero, chegamos a duas equações – as chamadas equações normais. Com duas incógnitas e duas equações, o sistema tem solução única.

$$\begin{cases} a \sum x_i^2 + b \sum x_i = \sum x_i y_i \\ a \sum x_i + nb = \sum y_i \end{cases}$$

Resolução do Sistema

Resolvendo o sistema diretamente, obtemos:

Coeficiente angular a:

$$a = \frac{n \sum x_i y_i - (\sum x_i) (\sum y_i)}{n \sum x_i^2 - (\sum x_i)^2}$$

Intercepto b:

$$b = \frac{\sum y_i - a \sum x_i}{n}$$

Reta ajustada:

$$\hat{y} = \left[\frac{n \sum x_i y_i - (\sum x_i) (\sum y_i)}{n \sum x_i^2 - (\sum x_i)^2} \right] x + b$$

Reta de Mínimos Quadrados

Com os valores de a e b encontrados, a reta que melhor ajusta os dados é:

$$\hat{y} = ax + b$$

Interpretação Geométrica

A reta $\hat{y} = ax + b$ minimiza a soma das distâncias verticais ao quadrado entre cada ponto observado (x_i, y_i) e o valor predito $\hat{y}_i$. É a melhor aproximação linear possível para os dados.

A Conexão entre Mínimos Quadrados e ANFIS

A estrutura dos consequentes do ANFIS é matematicamente idêntica à de uma regressão linear — quando os pesos normalizados $\bar{w}_i$ estão fixos. Isso é o que torna o LSE aplicável diretamente.

Analogia Direta

Regressão Linear	Consequente ANFIS
$\hat{y} = ax + b$	$f_i = p_i x + q_i y + r_i$
Parâmetros: a, b	Parâmetros: p_i, q_i, r_i

Nos dois casos, os coeficientes aparecem linearmente. Por isso, o LSE pode ser aplicado para encontrar p_i, q_i, r_i no ANFIS.

$$f = (\bar{w}_1 x)p_1 + (\bar{w}_1 y)q_1 + \bar{w}_1 r_1 + (\bar{w}_2 x)p_2 + (\bar{w}_2 y)q_2 + \bar{w}_2 r_2$$

Por que o Aprendizado Híbrido Funciona?

Parte Linear — LSE (Passo Forward)

Os pesos normalizados $\bar{w}_i$ são fixados. Os consequentes $\{p_i, q_i, r_i\}$ são encontrados por mínimos quadrados de forma exata e eficiente.

Parte Não-Linear — Backpropagation (Passo Backward)

Os consequentes são fixados. As funções de pertinência (premissas $\{a_i, b_i, c_i\}$) são ajustadas por gradiente descendente.

Resultado

- A separação entre parte linear e não-linear é a razão matemática pela qual o algoritmo híbrido converge muito mais rápido do que o gradiente descendente puro aplicado a todos os parâmetros simultaneamente.
- Os mínimos quadrados ajustam os consequentes; o backpropagation ajusta as funções de pertinência.

Calculando a Saída e o Erro do ANFIS

Com os parâmetros de consequente $\{p_1, q_1, r_1, p_2, q_2, r_2\}$ identificados pelo LSE no passo forward, calculamos a saída do ANFIS e o erro em relação ao valor desejado.

Consequentes Ajustados pelo LSE

Os consequentes de cada regra são:

$$f_1 = p_1x + q_1y + r_1$$

$$f_2 = p_2x + q_2y + r_2$$

A saída final do ANFIS é a média ponderada normalizada:

$$\hat{z} = \bar{w}_1 f_1 + \bar{w}_2 f_2$$

Erro e Backpropagation

Valor Desejado z^*

O valor de saída esperado para o par de entrada (x, y) , fornecido pelos dados de treinamento.

Erro Quadrático E

Comparamos a saída $\hat{z}$ com o valor desejado z^* e calculamos o erro:

$$E = \frac{1}{2}(z^* - \hat{z})^2$$

Passo Backward

É esse erro E que será retropropagado pela rede no passo backward. O gradiente $\partial E / \partial \theta$ é calculado para cada parâmetro de premissa $\theta \in \{a_i, b_i, c_i\}$, atualizando as funções de pertinência via gradiente descendente:

$$\theta \leftarrow \theta - \eta \frac{\partial E}{\partial \theta}$$

Regra da Cadeia — Como o Erro Chega às Funções de Pertinência

Para atualizar os parâmetros de premissa (como c de uma função de pertinência), precisamos saber como o erro E é afetado por c . Isso é feito pela regra da cadeia, propagando o gradiente através das camadas da rede. O erro percorre o seguinte caminho:

E Erro quadrático na saída	$\hat{z}$ Saída do ANFIS	w Força de disparo da regra
μ_A Grau de pertinência	c Parâmetro da função de pertinência	

$$\frac{\partial E}{\partial c} = \frac{\partial E}{\partial \hat{z}} \cdot \frac{\partial \hat{z}}{\partial w} \cdot \frac{\partial w}{\partial \mu_A} \cdot \frac{\partial \mu_A}{\partial c}$$

Interpretação de Cada Termo

- $\frac{\partial E}{\partial \hat{z}}$
Como o erro depende da saída $\hat{z}$. Derivada direta do erro quadrático: $-(z^* - \hat{z})$
- $\frac{\partial \hat{z}}{\partial w}$
Como a saída depende do peso da regra w_i . Relaciona a média ponderada com as forças de disparo.
- $\frac{\partial w}{\partial \mu_A}$
Como o peso da regra depende do grau de pertinência μ_A . Derivada do produto das pertinências.
- $\frac{\partial \mu_A}{\partial c}$
Como a pertinência depende do parâmetro c da função sino. Derivada da função de pertinência em relação ao seu centro.

Ideia Central
O erro é propagado de trás para frente através das camadas, multiplicando as derivadas parciais de cada etapa. Cada camada repassa sua parcela de responsabilidade pelo erro.

Parâmetros Atualizados
O mesmo processo se aplica a todos os parâmetros de premissa: $\{a_i, b_i, c_i\}$ de cada função de pertinência da Camada 1.

Cálculo de $\partial E / \partial \hat{z}$ — Derivada do Erro em Relação à Saída

O primeiro termo da regra da cadeia é a derivada do erro quadrático E em relação à saída $\hat{z}$ do ANFIS. É o ponto de partida do backpropagation.

Passo a Passo

Partimos da definição do erro quadrático:

$$E = \frac{1}{2}(z^* - \hat{z})^2$$

Aplicando a regra da cadeia (derivada de u^2 é $2u$):

$$\frac{\partial E}{\partial \hat{z}} = \frac{1}{2} \cdot 2(z^* - \hat{z}) \cdot \frac{\partial(z^* - \hat{z})}{\partial \hat{z}}$$

Como z^* é constante em relação a $\hat{z}$:

$$\frac{\partial(z^* - \hat{z})}{\partial \hat{z}} = -1$$

Portanto:

$$\frac{\partial E}{\partial \hat{z}} = -(z^* - \hat{z}) = \hat{z} - z^*$$

Interpretação

O sinal de $\partial E / \partial \hat{z}$ indica a direção do erro: se $\hat{z} > z^*$, a derivada é positiva (saída acima do desejado); se $\hat{z} < z^*$, é negativa (saída abaixo do desejado).

Caso Numérico

Se $z^* = 0.8$ e $\hat{z} = 0.5$, então:

$$\frac{\partial E}{\partial \hat{z}} = 0.5 - 0.8 = -0.3$$

Próximo Passo

Este gradiente é multiplicado pelos demais termos da regra da cadeia para propagar o erro até os parâmetros de premissa $\{a_i, b_i, c_i\}$ da Camada 1.

$$\frac{\partial E}{\partial c} = \underbrace{(\hat{z} - z^*)}_{\partial E / \partial \hat{z}} \cdot \frac{\partial \hat{z}}{\partial w} \cdot \frac{\partial w}{\partial \mu_A} \cdot \frac{\partial \mu_A}{\partial c}$$

Cálculo de $\partial \hat{z} / \partial w$ — Derivada da Saída em Relação à Força de Disparo

O segundo termo da regra da cadeia é a derivada da saída $\hat{z}$ em relação à força de disparo w_i de uma regra. Para isso, precisamos expandir a expressão da saída do ANFIS.

Passo a Passo

A saída do ANFIS é a média ponderada normalizada:

$$\hat{z} = \bar{w}_1 f_1 + \bar{w}_2 f_2 = \frac{w_1 f_1 + w_2 f_2}{w_1 + w_2}$$

Expandindo em termos de w_1 e w_2 (usando a regra do quociente para derivar em relação a w_1):

$$\frac{\partial \hat{z}}{\partial w_1} = \frac{f_1(w_1 + w_2) - (w_1 f_1 + w_2 f_2)}{(w_1 + w_2)^2}$$

Simplificando o numerador:

$$f_1(w_1 + w_2) - w_1 f_1 - w_2 f_2 = w_2 f_1 - w_2 f_2 = w_2(f_1 - f_2)$$

Portanto:

$$\frac{\partial \hat{z}}{\partial w_1} = \frac{w_2(f_1 - f_2)}{(w_1 + w_2)^2} = \frac{f_1 - \hat{z}}{w_1 + w_2}$$

De forma geral, para a regra i :

$$\frac{\partial \hat{z}}{\partial w_i} = \frac{f_i - \hat{z}}{w_1 + w_2}$$

Interpretação

$\partial \hat{z} / \partial w_i$ mede o quanto a saída $\hat{z}$ muda quando a força de disparo da regra i aumenta. Se $f_i > \hat{z}$, a derivada é positiva: aumentar w_i puxa a saída para cima.

Caso Numérico

Se $f_1 = 0.9$, $\hat{z} = 0.5$, $w_1 = 0.6$, $w_2 = 0.4$, então:

$$\frac{\partial \hat{z}}{\partial w_1} = \frac{0.9 - 0.5}{0.6 + 0.4} = \frac{0.4}{1.0} = 0.4$$

Próximo Passo

Este gradiente é multiplicado pelos demais termos para continuar a propagação do erro até μ_A e depois até c :

$$\frac{\partial E}{\partial c} = (\hat{z} - z^*) \cdot \underbrace{\frac{f_i - \hat{z}}{w_1 + w_2}}_{\partial \hat{z} / \partial w_i} \cdot \frac{\partial w_i}{\partial \mu_A} \cdot \frac{\partial \mu_A}{\partial c}$$

Cálculo de $\partial w_i / \partial \mu_A$ — Derivada da Força de Disparo em Relação à Pertinência

O terceiro termo da regra da cadeia é a derivada da força de disparo w_i em relação ao grau de pertinência μ_A . Para isso, precisamos lembrar como w_i é calculado na Camada 2.

Passo a Passo

Na Camada 2, a força de disparo da regra 1 é o produto das pertinências:

$$w_1 = \mu_{A_1}(x) \cdot \mu_{B_1}(y)$$

Derivando w_1 em relação a $\mu_{A_1}(x)$ — como $\mu_{B_1}(y)$ é constante em relação a μ_{A_1} :

$$\frac{\partial w_1}{\partial \mu_{A_1}} = \mu_{B_1}(y)$$

De forma análoga, para a regra 2:

$$w_2 = \mu_{A_2}(x) \cdot \mu_{B_2}(y) \implies \frac{\partial w_2}{\partial \mu_{A_2}} = \mu_{B_2}(y)$$

De forma geral, para a regra i :

$$\frac{\partial w_i}{\partial \mu_{A_i}} = \mu_{B_i}(y)$$

E de forma simétrica, derivando em relação a μ_{B_i} :

$$\frac{\partial w_i}{\partial \mu_{B_i}} = \mu_{A_i}(x)$$

Interpretação

$\partial w_i / \partial \mu_{A_i}$ mede o quanto a força de disparo da regra i muda quando a pertinência de A_i aumenta. O resultado é simplesmente a pertinência da outra variável (B_i), pois w_i é um produto.

Caso Numérico

Se $\mu_{A_1}(x) = 0.7$ e $\mu_{B_1}(y) = 0.6$, então $w_1 = 0.42$ e:

$$\frac{\partial w_1}{\partial \mu_{A_1}} = \mu_{B_1}(y) = 0.6$$

Próximo Passo

Com os três primeiros termos calculados, a regra da cadeia acumulada até aqui é:

$$\frac{\partial E}{\partial \mu_{A_i}} = (\hat{z} - z^*) \cdot \frac{f_i - \hat{z}}{w_1 + w_2} \cdot \mu_{B_i}(y)$$

Resta apenas calcular $\partial \mu_A / \partial c$ para completar a propagação até o parâmetro c da função sino.

Função Sino Generalizada — Derivada e Atualização

A função sino generalizada é a função de pertinência da Camada 1 do ANFIS. Para aplicar o backpropagation, precisamos calcular sua derivada em relação ao parâmetro c .

Função Sino Generalizada

Definida como:

$$\mu_A(x) = \frac{1}{1 + \left| \frac{x-c}{a} \right|^{2b}}$$

onde $\{a, b, c\}$ são os parâmetros de premissa: a controla a largura, b controla a inclinação das bordas e c é o centro da função.

A derivada em relação ao centro c é:

$$\frac{\partial \mu_A}{\partial c} = \frac{2b \left| \frac{x-c}{a} \right|^{2b}}{(x-c) \left(1 + \left| \frac{x-c}{a} \right|^{2b} \right)^2}$$

Com o gradiente calculado, c é atualizado por gradiente descendente:

$$c \leftarrow c - \eta \frac{\partial E}{\partial c}$$

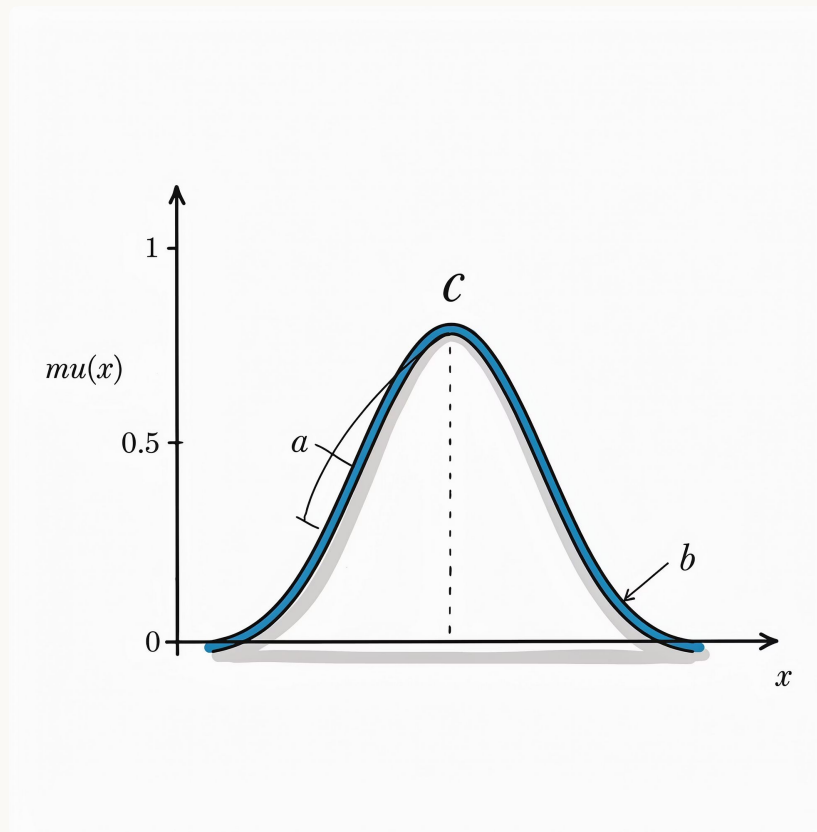


Figura 8. Função sino generalizada com parâmetros a (largura), b (inclinação) e c (centro).

Regra da Cadeia Completa — $\partial E / \partial c$

Reunindo todos os termos calculados nos slides anteriores, podemos escrever a derivada completa do erro E em relação ao parâmetro c da função sino generalizada.

$$\frac{\partial E}{\partial c} = \underbrace{(\hat{z} - z^*)}_{\partial E / \partial \hat{z}} \cdot \underbrace{\frac{f_i - \hat{z}}{w_1 + w_2}}_{\partial \hat{z} / \partial w_i} \cdot \underbrace{\mu_{B_i}(y)}_{\partial w_i / \partial \mu_{A_i}} \cdot \underbrace{\frac{2b \left| \frac{x-c}{a} \right|^{2b}}{(x-c) \left(1 + \left| \frac{x-c}{a} \right|^{2b} \right)^2}}_{\partial \mu_A / \partial c}$$

Resumo dos 4 Termos

① $\partial E / \partial \hat{z} = \hat{z} - z^*$

Erro entre a saída do ANFIS e o valor desejado. Ponto de partida do backpropagation.

② $\partial \hat{z} / \partial w_i = (f_i - \hat{z}) / (w_1 + w_2)$

Como a saída muda com a força de disparo da regra i. Obtido pela regra do quociente.

③ $\partial w_i / \partial \mu_{A_i} = \mu_{B_i}(y)$

Como a força de disparo muda com a pertinência de A_i . Resultado direto do produto das pertinências.

④ $\partial \mu_A / \partial c$

Derivada da função sino em relação ao centro c. Obtida pela regra da cadeia sobre a função sino generalizada.

Atualização Final de c

Com o gradiente completo calculado, o parâmetro c é atualizado por gradiente descendente a cada época:

$$c \leftarrow c - \eta \cdot \frac{\partial E}{\partial c}$$

Substituindo todos os termos:

$$c \leftarrow c - \eta \cdot (\hat{z} - z^*) \cdot \frac{f_i - \hat{z}}{w_1 + w_2} \cdot \mu_{B_i}(y) \cdot \frac{2b \left| \frac{x-c}{a} \right|^{2b}}{(x-c) \left(1 + \left| \frac{x-c}{a} \right|^{2b} \right)^2}$$

O mesmo vale para a e b

As derivadas $\partial \mu_A / \partial a$ e $\partial \mu_A / \partial b$ são calculadas de forma análoga, e os parâmetros a e b são atualizados pela mesma regra de gradiente descendente.

Convergência

Após cada época (forward + backward), os parâmetros $\{a_i, b_i, c_i\}$ se ajustam progressivamente, reduzindo o erro E até a convergência do modelo.

Backward Pass — Atualização dos Parâmetros de Premissa

Com o erro E calculado, o passo backward propaga o gradiente de volta pela rede até os parâmetros de premissa S_1 . Os parâmetros de consequente S_2 permanecem fixos durante este passo.

E	Backpropagation	S_1
Erro quadrático calculado no passo forward	Regra da cadeia propaga $\partial E / \partial \theta$ por todas as camadas	Parâmetros de premissa $\{a_i, b_i, c_i\}$ atualizados

Gradientes Calculados

Para cada função de pertinência, calculamos as três derivadas parciais:

$$\frac{\partial E}{\partial a_i}, \quad \frac{\partial E}{\partial b_i}, \quad \frac{\partial E}{\partial c_i}$$

Cada uma delas é obtida pela regra da cadeia completa:

$$\frac{\partial E}{\partial \theta} = \frac{\partial E}{\partial \hat{z}} \cdot \frac{\partial \hat{z}}{\partial w_i} \cdot \frac{\partial w_i}{\partial \mu_A} \cdot \frac{\partial \mu_A}{\partial \theta}$$

onde $\theta \in \{a_i, b_i, c_i\}$ é o parâmetro a ser atualizado.

Regra de Atualização

Cada parâmetro é atualizado por gradiente descendente:

$$\theta^{\text{novo}} = \theta^{\text{antigo}} - \eta \frac{\partial E}{\partial \theta}$$

Parâmetro c_i (centro)

$$c_i \leftarrow c_i - \eta \cdot (\hat{z} - z^*) \cdot \frac{f_i - \hat{z}}{w_1 + w_2} \cdot \mu_{B_i}(y) \cdot \frac{\partial \mu_A}{\partial c_i}$$

Parâmetros a_i e b_i

Calculados de forma análoga, substituindo $\partial \mu_A / \partial c$ pelas respectivas derivadas $\partial \mu_A / \partial a$ e $\partial \mu_A / \partial b$ da função sino generalizada.

S_2 permanece fixo

Durante o passo backward, os parâmetros de consequente $\{p_i, q_i, r_i\}$ não são alterados. Eles serão re-estimados pelo LSE na próxima época (passo forward).

Nova Época — O Ciclo se Repete

Após o backward pass, as funções de pertinência foram modificadas. Isso altera os pesos de disparo, que por sua vez exigem que os parâmetros de consequente sejam recalculados. Uma nova época começa.

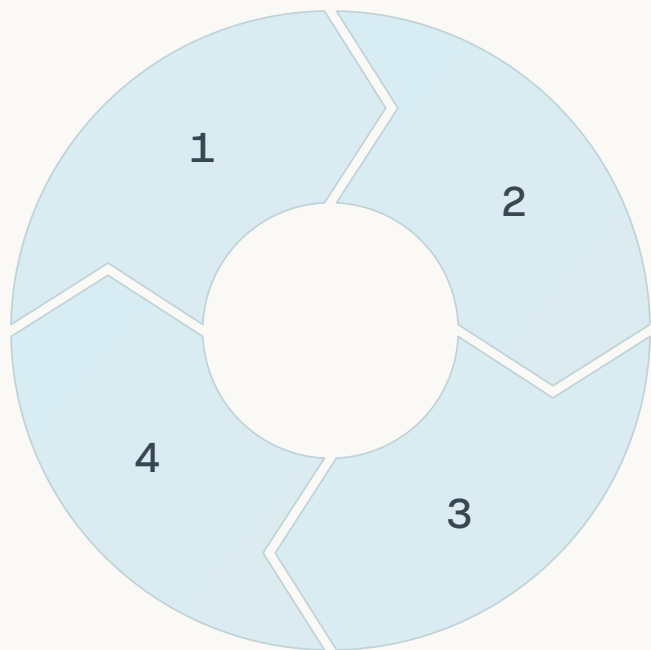
Novas Funções de Pertinência

Os parâmetros $\{a_i, b_i, c_i\}$ foram atualizados pelo backward pass. As curvas $\mu_{A_i}(x)$ assumem novas formas.

Novo Erro e Backpropagation

Calculamos $\hat{z}$ com os novos consequentes, obtemos o novo erro E e realizamos outro backward pass. O ciclo continua até a convergência.

$$E^{(\text{novo})} = \frac{1}{2} (z^* - \hat{z}^{(\text{novo})})^2$$



Novos Pesos de Disparo

Com as pertinências modificadas, as forças de disparo são recalculadas:

$$w_i^{(\text{novo})} = \mu_{A_i}^{(\text{novo})}(x) \cdot \mu_{B_i}^{(\text{novo})}(y)$$

E os pesos normalizados:

$$\bar{w}_i^{(\text{novo})} = \frac{w_i^{(\text{novo})}}{w_1^{(\text{novo})} + w_2^{(\text{novo})}}$$

Novos Mínimos Quadrados (LSE)

Com os novos pesos $\bar{w}_i$ fixados, os consequentes são re-estimados pelo LSE:

$$\min_{\theta} \sum_{k=1}^N \left(f_k^* - \hat{f}(x_k, \theta) \right)^2$$

Obtendo novos parâmetros p_i, q_i, r_i .

Esse ciclo forward → backward se repete por todas as épocas até que o erro E convirja para um valor mínimo aceitável.

Variantes do Algoritmo Híbrido ANFIS

O algoritmo híbrido pode ser aplicado de duas formas distintas, dependendo de como os dados de treinamento são processados: em lote (batch) ou de forma online (estocástica).

ANFIS Híbrido Clássico (Batch)

Todas as amostras são processadas de uma vez a cada época. O passo forward usa o conjunto completo:

Passo Forward (todas as amostras)

Calcula w_i e $\bar{w}_i$ para todas as amostras. Resolve o LSE globalmente:

$$\{x_k, y_k\}_{k=1}^N \rightarrow w_i, \bar{w}_i \rightarrow \text{LSE} \rightarrow p_i, q_i, r_i$$

Passo Backward (todas as amostras)

Calcula o erro acumulado e os gradientes sobre todo o conjunto. Atualiza os parâmetros de premissa:

$$\{x_k, y_k\}_{k=1}^N \rightarrow E \rightarrow \nabla E \rightarrow \text{Atualização de } \{a_i, b_i, c_i\}$$

Quando usar o Clássico?

Dados disponíveis todos de uma vez. Convergência mais estável. Corresponde à descrição original do algoritmo híbrido de Jang (1993).

ANFIS Online / Estocástico

Cada amostra é processada individualmente à medida que chega. Ideal quando os dados chegam continuamente, como no exemplo do ventilador.

Amostra k chega

Uma nova observação $(x_k, y_k) \rightarrow z_k^*$ é recebida.

$$k = 1, 2, 3, \dots$$

Forward + Erro

Calcula $w_i, \bar{w}_i$, consequentes e a saída $\hat{z}_k$. Obtém o erro da amostra:

$$E_k = \frac{1}{2} (z_k^* - \hat{z}_k)^2$$

Gradiente + Atualização

Calcula os gradientes e atualiza imediatamente os parâmetros de premissa:

$$\theta \leftarrow \theta - \eta \frac{\partial E_k}{\partial \theta}$$

Quando usar o Online?

Dados chegam em tempo real (sensores, streaming). Adaptação contínua do modelo. Útil em sistemas de controle como o ventilador.

Exercício Prático

Atividade Prática — Implementação de um Sistema ANFIS

Objetivo

Implementar, em Python, um sistema **ANFIS (Adaptive Neuro-Fuzzy Inference System)** utilizando um modelo **Fuzzy Sugeno de primeira ordem**, aplicado à estimativa da velocidade de um ventilador a partir da **temperatura** e da **umidade relativa do ar**.

O treinamento deverá utilizar o **aprendizado híbrido**, combinando:

- **Método dos Mínimos Quadrados** para ajuste dos parâmetros consequentes;
- **Backpropagation**, utilizando gradiente descendente, para ajuste dos parâmetros das funções de pertinência.

Referências Bibliográficas

- **JANG, Jyh-Shing Roger; SUN, Chuen-Tsai; MIZUTANI, Eiji.** *Neuro-fuzzy and soft computing: a computational approach to learning and machine intelligence*. Upper Saddle River, NJ: Prentice Hall, 1997. 614 p.
- **HAYKIN, S.** Neural Networks and Learning Machines. 3ª ed. Pearson.
- **GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A.** Deep Learning. MIT Press, 2016.